

Chapter 6 - If-statements

We use the code from the previous chapter

With if-statements, we can allow pieces of code to only be executed when a condition is met.

You create an if-statement like this:

```
if condition then
```

```
    -- code
```

```
end
```

A condition, or statement, is something that's either true or false.

For example: $5 > 9$

The $>$ means, higher than. So the statement is that 5 is higher than 9, which is false.

Put an if-statement around the code of x increasing.

```
function love.update(dt)
```

```
    if  $5 > 9$  then
```

```
         $x = x + 100 * dt$ 
```

```
    end
```

```
end
```

When you run the game you'll notice that our rectangle isn't moving. This is because the statement is false. If we were to change the statement to $5 < 9$ (5 is lower than 9), then the statement is true, and the code inside the if-statement will execute.

With this, we can for example make x go up to 600, and then make it stop moving, with $x < 600$.

```
function love.update(dt)
```

```
    if  $x < 600$  then
```

```
         $x = x + 100 * dt$ 
```

end

end



If we want to check if a value is equal to another value, we need to use 2 equal-signs (==).

For example: `4 == 7`

1 equal-sign is for assigning, 2 equal-signs is for comparing.

`x = 10` --Assign 10 to x

`x == 10` --Compare 10 to x

We can also use `>=` and `<=` to check if values are higher or equal to each other or if the values are lower and equal to each other.

`10 <= 10` --true, 10 equals to 10

`15 >= 4` --true, 15 is higher than 4

So the code above is a shorthand for

```
10 == 10 or 10 < 10
```

```
15 == 4 or 15 > 4
```

Boolean

A variable can also be true or false. This type of variable is what we call a boolean.

Let's make a new variable called move, with the value true, and check if move is true in our if-statement.

```
function love.load()
```

```
    x = 100
```

```
    move = true
```

```
end
```

```
function love.update(dt)
```

```
    -- Remember, 2 equal signs!
```

```
    if move == true then
```

```
        x = x + 100 * dt
```

```
    end
```

```
end
```

move is true, so our rectangle moves. But move == true is actually redundant. We're checking if it's true that the value of move is true. Simply using move as a statement is good enough.

```
if move then
```

```
    x = x + 100 * dt
```

```
end
```

If we want to check if move is false, we can put a not in front of it.

```
if not move then
```

```
    x = x + 100 * dt
```

```
end
```

If we want to check if a number is NOT equal to another number, we use a tilde (~).

```
if 4 ~= 5 then
```

```
    x = x + 100 * dt
```

```
end
```

We can also assign true or false to a variable with a statement.

For example: move = 6 > 3.

If we check if move is true, and then change move to false inside the if-statement, it's not as if we jump out of the if-statement. All the code below will still be executed.

```
if move then
```

```
    move = false
```

```
    print("This will still be executed!")
```

```
    x = x + 100 * dt
```

```
end
```

Arrow keys

Let's make the rectangle move based on if we hold down the right arrowkey. For this we use the function love.keyboard.isDown ([wiki](#)).

Notice how the D of Down is uppercase. This type of casing is what we call camelCasing. We start the first word in lowercase, and then every following word's first character we type in uppercase. This type of casing is also what we will be using for our variables throughout this tutorial.

We pass the string "right" as first argument to check if the right arrowkey is down.

```
if love.keyboard.isDown("right") then
```

```
    x = x + 100 * dt
```

```
end
```

So now only when we hold down the right arrowkey does our rectangle move.



We can also use else to tell our game what to do when the condition is false. Let's make our rectangle move to the left, when we don't press right.

```
if love.keyboard.isDown("right") then
```

```
    x = x + 100 * dt
```

```
else
```

```
    x = x - 100 * dt --We decrease the value of x
```

```
end
```

We can also check if another statement is true, after the first is false, with elseif. Let's make it so that after checking if the right arrowkey is down, and it's not, we'll check if the left arrowkey is down.

```
if love.keyboard.isDown("right") then
```

```
    x = x + 100 * dt
```

```
elseif love.keyboard.isDown("left") then
```

```
    x = x - 100 * dt
```

end

Try to make the rectangle move up and down as well.

and & or

With and we can check if multiple statements are true.

if 5 < 9 and 14 > 7 then

 print("Both statements are true")

end

With or, the if-statement will execute if any of the statements is true.

if 20 < 9 or 14 > 7 or 5 == 10 then

 print("One of these statements is true")

end

One more thing

To be more precise, if-statements check if the statement is NOT false or nil.

if true then print(1) **end** *--Not false or nil, executes.*

if false then print(2) **end** *--False, doesn't execute.*

if nil then print(3) **end** *--Nil, doesn't execute*

if 5 then print(4) **end** *--Not false or nil, executes*

if "hello" then print(5) **end** *--Not false or nil, executes*

--Output: 1, 4, 5

Summary

With if-statements, we can allow pieces of code to only be executed when a condition is met. We can check if a number is higher, lower or equal to another number/value. A variable can be true or false. This type of variable is what we call a boolean. We can use

else to tell our game what to execute when the statement was false, or elseif to do another check.